The Evolution of AI Architecture

Slide Title: From Script to System: Architecting AI Applications & The User Feedback Loop

Visual Suggestion: A schematic diagram evolving from a simple circle (User) $\leftrightarrow$ Box (Model) into a complex network of Gateways, Caches, and Feedback Loops.

Speaker Notes: "Welcome, everyone. We've spent the last decade perfecting our models. Now, we have to perfect the systems that hold them. Today, based on Chip Huyen's final chapter in *AI Engineering*, we are moving beyond 'prompt engineering' to true 'system engineering'. We will dissect the architecture of a mature AI platform—step by step—and then dive into the most critical asset you possess: your user feedback data. Let's explore how to build systems that don't just answer queries but actually learn from them."

The Starting Point: The MVP Architecture

Slide Title: The "Hello World" of AI Apps

- **Key Content:**

- **The Simplest Form:** User sends query -> Model generates response -> User receives response .

- **The Reality:** No context, no guardrails, no optimization.

- **The Limitation:** This works for a demo, but fails in production due to hallucinations, latency, and safety risks.

- **Visual Suggestion:** A minimalist diagram (based on Figure 10-1): A stick figure user connected by a single line to a box labeled "Model API" .

- **Speaker Notes:** "Every AI engineer starts here. It’s the 'Hello World' of LLMs. You send a query, you get an answer. It feels magical. But in production, this architecture is a ticking time bomb. It has no memory, no safety net, and no cost control. This chapter is about the journey from this simple box to a resilient platform. We’re going to layer five critical components on top of this foundation."

Step 1: Context Construction (RAG & Agents)

Slide Title: Context is King: RAG and Tool Use

- **Key Content:**
  - **The Goal:** Provide the model with the necessary information to answer the query (Feature Engineering for LLMs).
  - **Mechanisms:** Retrieval (text/image/tabular) and Tools (web search, APIs).
  - **Variability:** Providers differ vastly in support; generic APIs have file limits, while specialized RAG solutions scale infinitely.
- **Visual Suggestion:** Update the previous diagram (based on Figure 10-2) to include a "Context Construction" box fed by "Read-only actions" and "Databases" feeding into the Model API .
- **Speaker Notes:** "The first upgrade is memory. A model without context is just a hallucination machine. We add 'Context Construction'—this is essentially feature engineering for Foundation Models. Whether you are retrieving a PDF from a vector DB or using a tool to check the weather, you are constructing the prompt dynamically. But be warned: not all APIs are built equal. A generic API might choke on 10 files, while a dedicated RAG pipeline can handle millions."

Step 2: Input Guardrails (Defense First)

Protecting the System: Input Guardrails

•**Key Content:**

•**The Risk:** Leaking private info (PII) to external APIs or executing malicious prompt injections.
•**The Leak Vectors:** Employees copying secrets, developers hardcoding internal policies, or tools retrieving sensitive DB rows .

•**The Fix:** PII Redaction/Masking. Replace sensitive data with placeholders like *[PHONE_NUMBER] before sending to the model.*

•**Visual Suggestion:** A flow diagram (Figure 10-3) showing a query with a "secret token" being masked into *[ACCESS_TOKEN] before hitting the model, and unmasked afterward .*

•**Speaker Notes:** "Before we let the model speak, we must ensure the user isn't dangerous—and that we aren't dangerous to the user. Input guardrails are non-negotiable. We've all heard the horror stories of employees pasting company secrets into chatbots. We need automatic PII redaction. We swap real secrets for placeholders before the data ever leaves our perimeter. It’s a simple masking technique that saves you from a compliance nightmare."

Slide 5: Step 2: Output Guardrails (Quality Control)

Slide Title: Protecting the User: Output Guardrails

- **Key Content:**

- **The Purpose:** Catch failures (empty responses, malformed JSON) and policy violations (toxicity, bad advice) .

- **Retry Logic:** If a response fails (e.g., bad JSON), try again. Models are probabilistic; a second try might work .

- **Latency Trade-off:** Sequential retries double latency. Parallel calls (generate 2, pick 1) reduce latency but increase cost.

- **Visual Suggestion:** A decision tree: Output -> Check Guardrail -> Fail? -> Retry/Fallback to Human.

- **Speaker Notes:** "Output guardrails are your quality assurance. Did the model return valid JSON? Did it start spouting racist nonsense? If it fails, what's your policy? You can retry—simply asking again often fixes the glitch. But retries kill latency. A pro tip from the book: if latency is key, send two requests in parallel and pick the winner. It costs twice as much, but it keeps your user experience snappy."

Slide 6: The Guardrail vs. Latency Dilemma

Slide Title: The Safety-Speed Trade-off

- **Key Content:**
  - **The Conflict:** Guardrails add latency. Some teams skip them to prioritize speed, which is risky .
  - **Streaming Challenge:** In streaming mode, unsafe tokens might reach the user before the guardrail catches them.
  - **Architecture Update:** Place guardrails at the Model Gateway or Inference Service level for centralization.
- **Visual Suggestion:** A scale balancing "Safety" (Guardrails) and "Speed" (Latency). A warning icon for "Streaming Mode."
- **Speaker Notes:** "Here is the uncomfortable truth: Guardrails slow you down. I've talked to teams who disable them because they can't afford the 500ms delay. That is a dangerous game. Also, streaming makes this harder—if you stream tokens, the user might see the insult before your toxicity filter catches it. You have to design your UX to handle retrospective retraction or accept the risk."

Step 3: The Model Router

Slide Title: The Traffic Controller: Model Routing

- **Key Content:**
  - **Concept:** Don't use GPT-4 for everything. Route simple queries to cheaper models and complex ones to capable models.
  - **Intent Classification:** Predict user intent (e.g., reset password vs. troubleshooting) to route to FAQs vs. Models vs. Humans .
  - **Implementation:** Use small models (BERT, Llama-7B) as the router. They must be fast and cheap.

- **Visual Suggestion:** A router node splitting traffic: "Password Reset" -> FAQ Database; "Complex Code" -> GPT-4; "Billing" -> Human Agent.

- **Speaker Notes:** "One model does not fit all. Using a massive model to answer 'how do I reset my password' is like driving a Ferrari to the mailbox. It's wasteful. We introduce a Router—usually a tiny, fast model like BERT—to classify intent. If it's a simple FAQ, send it to a database. If it's complex reasoning, send it to the big model. This is how you cut costs by 50% without hurting quality."

Slide 8: Step 3: The Model Gateway

Slide Title: The Unified Interface: Model Gateway

- **Key Content:**
  - **Definition:** An intermediate layer unifying access to all models (OpenAI, Anthropic, Self-hosted) .
  - **Benefits:**
    - **Security:** Centralized API key management (no keys in code).
    - **Resilience:** Fallback policies for rate limits or API outages.
    - **Maintainability:** Swap providers without rewriting application code.
- **Visual Suggestion:** Diagram (Figure 10-6) showing various apps connecting to one "Model Gateway" which then branches out to OpenAI, Google, and Self-Hosted models .
- **Speaker Notes:** "You need a Gateway. Trust me. Do not hardcode OpenAI keys into your five different microservices. A Gateway provides a single, unified API for your organization. It handles access control, cost tracking, and most importantly, fallbacks. When an API goes down—and it will—the gateway can silently switch to a backup model without your users ever knowing."

Step 4: Exact Caching

Slide Title: The Quick Win: Exact Caching

- **Key Content:**
  - **Mechanism:** If a user asks the exact same question, return the stored answer. Don't re-compute .
  - **Use Cases:** Multi-step reasoning chains, SQL execution, or embedding generation.
  - **Eviction Policy:** Use LRU (Least Recently Used) or Time-to-Live. Don't cache time-sensitive queries like "What's the weather?".

- **Visual Suggestion:** A flowchart: Query -> Cache Hit? -> Yes (Return Result) / No (Generate & Store).

- **Speaker Notes:** "The fastest generation is no generation. Caching is old tech, but vital. If user A asks for a product summary, save it. When user B asks for the same summary 10 seconds later, serve the cache. It saves money and latency. Just be careful with time-sensitive data—you don't want to serve yesterday's weather report."

Slide 10: Step 4: Semantic Caching (The Danger Zone)

Slide Title: The Risky Win: Semantic Caching

•**Key Content:**

•**Concept:** Reuse answers for queries that are *similar* but not identical (e.g., "Capital of Vietnam?" vs. "What is Vietnam's capital?") .
•**Implementation:** Vector search. If similarity score > threshold, return cached answer .
•**The Risk:** High false positives. If the threshold is wrong, you serve wrong answers. It also requires an expensive vector search.

•**Data Leak Risk:** Caching a "generic" query that actually contains user-specific context (e.g., "return policy" linked to a specific user tier) .

•**Visual Suggestion:** A diagram comparing two query vectors in space. If distance < X, link them to the same cached response.

•**Speaker Notes:** "Semantic caching is the 'danger zone'. It sounds great—serving the same answer for 'What's the price?' and 'How much does it cost?'. But it relies on vector similarity thresholds. If you get it wrong, you serve nonsense. Worse, you risk data leaks. If User A asks about 'my order' and you cache it, User B might see User A's order details. Use this with extreme caution."

Slide 11: Step 5: Agentic Patterns

Slide Title: Closing the Loop: Agentic Patterns

- **Key Content:**

- **Shift from Linear to Loops:** Instead of Query -> Response, the system can Loop: Generation -> Evaluation -> Re-retrieval .

- **Capability:** Allows the system to self-correct if the initial retrieval was insufficient.

- **Visual:** Figure 10-9 showing a feedback loop from "Response" back into "Context Construction" .

- **Visual Suggestion:** A circular flow diagram showing the model output feeding back into the input for a second pass.

- **Speaker Notes:** "Now we get complex. We move from linear pipelines to Agentic Loops. If the model generates an answer and realizes 'Wait, I'm missing info,' it can loop back, trigger a new search, update the context, and try again. This is how we solve complex tasks, but it introduces the risk of infinite loops."

Slide 12: Step 5: Write Actions (High Stakes)

Slide Title: Crossing the Rubicon: Write Actions

- **Key Content:**
  - **Definition:** Allowing the model to change the environment (Send email, API POST, Buy stock) .
  - **Risk Profile:** Vastly higher risk. Hallucinations become irreversible actions.
  - **Requirement:** Requires strict "Human-in-the-loop" approval flows for sensitive actions.

- **Visual Suggestion:** Figure 10-10, highlighting the "Write Actions" box (e.g., Update Orders) in orange/red to signify risk .

- **Speaker Notes:** "The final architectural step is 'Write Actions'. This is where the AI stops just talking and starts *doing*. Sending emails, booking flights, updating databases. This unlocks massive value, but the risk profile explodes. A hallucination here isn't a typo; it's a lawsuit. You need bulletproof guardrails here."

Slide 13: Orchestration

Slide Title: Taming Complexity: Orchestration

- **Key Content:**

- **Role:** The glue that chains retrieval, prompting, generation, and guardrails together.
- **Two Functions:** Component definition (what tools/models exist) and Chaining (the execution flow).
- **Build vs. Buy:** Start without one. Only add tools like LangChain/LlamaIndex when complexity demands it. Orchestrators can abstract away critical debugging details .

- **Visual Suggestion:** A pipeline diagram: Step 1 (Process) -> Step 2 (Retrieve) -> Step 3 (Generate) -> Step 4 (Eval).

- **Speaker Notes:** "How do you manage this mess of routers, caches, and models? Orchestrators. They define the chain of events. Tools like LangChain are popular, but a word of warning: don't start with them. Start simple. Orchestrators abstract away the details, and when things break—and they will—you need to understand what's happening under the hood."

Slide 14: Observability vs. Monitoring

Slide Title: Observability: Seeing Inside the Black Box

•**Key Content:**

•**Monitoring:** "The system is failing." (Looking at external outputs) .
•**Observability:** "Why is the system failing?" (Inferring internal state from outputs) .
•**Key DevOps Metrics:** MTTD (Mean Time to Detection), MTTR (Mean Time to Response), CFR (Change Failure Rate) .

•**Visual Suggestion:** An iceberg. "Monitoring" is the tip above water. "Observability" is the deep understanding of the mass below.

•**Speaker Notes:** "You can't fix what you can't see. We distinguish between Monitoring and Observability. Monitoring tells you *that* you are failing. Observability tells you *why*. In AI, where failures are often nondeterministic and subtle, you need deep observability. You need to know not just that the user downvoted, but what retrieval chunks were used and what the temperature setting was."

Slide 15: Drift Detection

Slide Title: The Silent Killer: Data Drift

- **Key Content:**
  - **System Prompt Drift:** Accidental changes to templates or typos.
  - **User Behavior Drift:** Users adapt to the model (e.g., learning to write shorter prompts), skewing metrics .
  - **Model Drift:** API providers update models behind the scenes (e.g., GPT-4 March vs. June versions), often causing performance drops .

- **Visual Suggestion:** A line graph showing "Model Performance" dropping suddenly due to a "Hidden API Update."

- **Speaker Notes:** "Even if you touch nothing, your system will degrade. Why? Drift. Maybe OpenAI updated the model behind the scenes. Maybe your users learned to game the prompt. Maybe a developer fixed a typo in the system prompt that broke the logic. You need to monitor for these silent shifts."

Part 2: User Feedback & The Data Flywheel (Slides 16-30)

The Data Flywheel

Slide Title: User Feedback: The Ultimate Moat

- **Key Content:**

- **Value:** User feedback is proprietary data. It creates a "Data Flywheel": more usage -> more data -> better models -> more usage.

- **Competitive Advantage:** Data is scarce. Proprietary interaction data is the only thing competitors cannot copy.

- **Visual Suggestion:** A flywheel graphic spinning. Arrows: Usage -> Feedback -> Improvement -> Usage.

- **Speaker Notes:** "Now, let's talk about your most valuable asset. It's not your code. It's not your prompt. It's your user data. Feedback creates a flywheel. The more people use your product, the smarter it gets, and the harder it is for a competitor to catch up. This is why you must obsess over feedback collection."

Slide 17: Explicit vs. Implicit Feedback

Slide Title: Asking vs. Observing

- **Key Content:**
  - **Explicit:** Thumbs up/down, star ratings, written reviews. High signal, low volume (requires user effort).
  - **Implicit:** Inferred from actions (clicks, retention, sharing). Low signal, high volume (abundant).
  - **Conversational Interfaces:** Unique because users can give natural language feedback directly in the workflow.

- **Visual Suggestion:** A comparison table. Explicit: "Clean, Sparse." Implicit: "Noisy, Abundant."

- **Speaker Notes:** "We have two buckets. Explicit feedback—like a thumbs up—is clean but rare. Users are lazy; they won't click unless they are thrilled or furious. Implicit feedback is messy but everywhere. Did they copy the code? Did they share the chat? In conversational AI, we have a superpower: the user tells us what they think in plain English."

Slide 18: Natural Language Signals: Early Termination

Slide Title: Signal: "Stop!" (Early Termination)

- **Key Content:**
  - **The Signal:** User stops generation, exits app, or stops responding.
  - **Meaning:** The conversation is going poorly.
  - **Action:** Track how often users interrupt the model. High interruption rates = low satisfaction.

- **Visual Suggestion:** A chat interface showing a user clicking a "Stop Generating" button mid-sentence.

- **Speaker Notes:** "When a user hits 'Stop Generating', they are shouting 'You are wasting my time.' This is a powerful negative signal. If your 'Stop' rate is climbing, your latency is too high or your quality is too low."

Slide 19: Natural Language Signals: Error Correction

Slide Title: Signal: "No, I meant..." (Error Correction)

- **Key Content:**
- **The Signal:** Messages starting with "No," "Actually," or rephrased queries.
- **Example:** User asks for "Mission Bay weather" $\rightarrow$ Model gives SF $\rightarrow$ User corrects "Mission Bay San Diego" .
- **Action:** Use these pairs (bad response + correction) as negative training examples.
- **Visual Suggestion:** Screenshot of Figure 10-12 (Weather example) showing the rephrase loop.
- **Speaker Notes:** "When a user corrects the model, they are doing your work for you. They are labeling the data. 'No, I meant the other Mission Bay.' This is gold. It tells you exactly where the ambiguity lies and how to fix the routing."

Slide 20: Natural Language Signals: Sentiment & Refusals

Slide Title: Signal: Sentiment and Refusals

- **Key Content:**
  - **Sentiment:** Expressions of frustration ("Ugh", "You are wrong") or gratitude.
  - **Refusal Rate:** If the model says "As an AI, I cannot...", the user is likely unhappy .
  - **Analysis:** Monitor the sentiment arc of a conversation (e.g., angry start -> happy end = success).
- **Visual Suggestion:** A sentiment trend line graph overlaying a chat log.
- **Speaker Notes:** "Track the 'I'm sorry' rate. If your model apologizes constantly, your guardrails are too tight. Also, look at sentiment. A conversation that starts angry and ends happy is a triumph. A conversation that ends with 'Ugh' is a failure."

Slide 21: Behavioral Signals: Regeneration

Slide Title: Signal: The "Retry" Button (Regeneration)

- **Key Content:**
  - **The Signal:** User clicks "Regenerate".
  - **Ambiguity:** Could mean the first answer was bad, OR the user just wants variety (common in creative tasks) .
  - **Billing Impact:** Usage-based billing reduces "idle curiosity" regenerations, making the signal stronger.

- **Visual Suggestion:** A UI button "Regenerate" with a question mark "Dissatisfaction or Curiosity?".

- **Speaker Notes:** "Regeneration is tricky. In Midjourney, I regenerate to see cool variations. In ChatGPT, I regenerate because the first answer was wrong. You must understand your domain. If you charge per token, a regeneration is a very strong negative signal—the user is paying twice to get it right."

Slide 22: Behavioral Signals: Organization

Slide Title: Signal: Copy, Share, Delete

- **Key Content:**
  - **Delete:** Strong negative signal (unless privacy-related).
  - **Rename:** Positive signal (user cares enough to organize it).
  - **Copy/Edit:** If a user edits code, the original was "losing" and the edit is "winning" preference data.

- **Visual Suggestion:** Icons for "Copy", "Delete", "Edit" with "Good/Bad" indicators.

- **Speaker Notes:** "Look at the meta-actions. Deleting a chat usually means 'garbage'. Renaming it means 'keeper'. If a user copies your code output and edits it in the IDE, capture that edit! That delta is the difference between your model and the truth."

Slide 23: Feedback Design: Timing is Everything

Slide Title: When to Ask for Feedback

- **Key Content:**
  - **The Rule:** Non-intrusive. Don't block workflow.
  - **Key Moments:**
    - **Start:** Calibration (e.g., FaceID setup).
    - **Failure:** When the model refuses or users regenerate.
    - **Low Confidence:** When the model isn't sure, show options.
- **Visual Suggestion:** A timeline of a user journey highlighting "Calibration" and "Error" points as feedback opportunities.
- **Speaker Notes:** "Don't annoy your users. Ask for feedback only when it matters. Ask at the start for calibration. Ask when they are frustrated. And ask when the model itself is unsure. This is 'uncertainty sampling'—asking humans to label the hardest examples."

Slide 24: UX Patterns: Human-AI Collaboration

Slide Title: UX Pattern: Inpainting and Collaboration

- **Key Content:**
  - **Collaborative Correction:** Let users fix specific mistakes instead of regenerating everything.
  - **Example:** DALL-E Inpainting. User selects a region (e.g., a sad cat) and prompts "Make it happy" .
  - **Benefit:** High-quality, granular feedback data.

- **Visual Suggestion:** Figure 10-14 showing the DALL-E inpainting workflow on a cat image.

- **Speaker Notes:** "The best feedback mechanism looks like a feature. DALL-E's inpainting is brilliant. It lets the user fix just the eyes of the cat without redrawing the tail. For the user, it's control. For you, it's granular, localized error data."

Slide 25: UX Patterns: Comparative Feedback

Slide Title: UX Pattern: Side-by-Side Comparison

- **Key Content:**
  - **Concept:** Show two responses, ask user to pick the best.
  - **Value:** Gold standard for RLHF (Reinforcement Learning from Human Feedback).
  - **Challenge:** Cognitive load. Users hate reading two essays.
  - **Solution:** Google Gemini method—show partial drafts to lower effort .
- **Visual Suggestion:** Figure 10-15 (ChatGPT side-by-side) and Figure 10-16 (Gemini partial drafts) .
- **Speaker Notes:** "Asking 'Is this good?' is hard. Asking 'Is A better than B?' is easy. Comparative feedback is the fuel for RLHF. But don't make them read two novels. Gemini collapses the drafts so users just click the one that looks promising. Reduce the friction."

Slide 26: UX Patterns: Integrated Feedback

Slide Title: UX Pattern: The "Ghost Text" (Copilot)

•**Key Content:**

- •**Method:** Feedback integrated into the core action.
- •**Example:** GitHub Copilot. "Tab" to accept = Positive. Keep typing to ignore = Negative.
- •**Advantage:** Zero friction. Every interaction is a label.

•**Visual Suggestion:** Figure 10-19 showing Copilot's "ghost text" code suggestion .

•**Speaker Notes:** "GitHub Copilot has the best feedback loop in history. You press Tab, they get a +1. You keep typing, they get a -1. It is invisible, frictionless, and captures every single keystroke. This is the holy grail of feedback design."

Slide 27: The Bias Trap: Leniency & Randomness

Slide Title: Data Hazard: Feedback Biases

•**Key Content:**

- •**Leniency Bias:** Users rate positively to be "nice" or avoid friction. (Uber 4.8 star average) .
- •**Randomness:** Users clicking randomly to make the popup go away.
- •**Position Bias:** Users prefer the first option shown.

•**Visual Suggestion:** A graph showing a skewed rating distribution (mostly 5 stars) labeled "Leniency Bias."

•**Speaker Notes:** "Data isn't truth; it's behavior. Users lie. They give 5 stars because they feel bad for the Uber driver. They click the first option because they are lazy (Position Bias). If you train on this raw data without de-biasing, you train a biased model."

Slide 28: Degenerate Feedback Loops

Slide Title: The "Cat Photo" Problem: Degenerate Loops

- **Key Content:**
  - **Definition:** Predictions influence feedback, which influences the next model, amplifying bias.
  - **Scenario:**
    1. Users like cat photos.
    2. System shows more cats.
    3. Cat lovers flock to app.
    4. App becomes only about cats. .
  - **Sycophancy:** Models learn to agree with the user rather than tell the truth.

- **Visual Suggestion:** A loop diagram showing "Bias" amplifying with each cycle until the output is homogeneous.

- **Speaker Notes:** "Be careful what you optimize for. If you optimize purely for clicks, you get clickbait. This is the 'Degenerate Feedback Loop.' If your model agrees with the user just to get a thumbs up, it becomes a sycophant—a liar. It will reinforce the user's wrong beliefs just to get a reward. You must balance feedback with ground truth."

Slide 29: Privacy and Data Donation

Slide Title: Ethics: Privacy & Data Donation

•**Key Content:**

•**The Problem:** Standalone apps (ChatGPT) lack context on how output is used (e.g., was the email sent?).

•**Context Requirement:** To debug, you need the conversation history, which is sensitive.

•**Solution:** Data Donation. Explicitly ask users to "Share chat for debugging" when they report a bug .

•**Visual Suggestion:** A mockup of a "Send Feedback" modal with a checkbox: "Include conversation history to help us debug."

•**Speaker Notes:** "You need context to fix bugs, but context is private. You can't just spy on users. The solution is 'Data Donation.' When a user complains, ask them: 'Can we see the last 5 messages to fix this?' Most will say yes. It respects privacy while giving you the data you need.

Slide 30: Conclusion

Slide Title: Conclusion: System Thinking

•**Key Content:**

•**Summary:** We moved from a simple API call to a complex system of Guardrails, Routers, Gateways, and Feedback Loops.

•**Takeaway:** AI challenges are System Problems. Solution requires looking at the whole architecture.

•**Final Thought:** "The hardest part isn't finding answers, but asking the right questions.".

•**Visual Suggestion:** The final, complex architecture diagram (Figure 10-10) fully assembled.

•**Speaker Notes:** "To wrap up: AI engineering is no longer just about the model. It is about the system. It is about the router that saves you money, the guardrail that saves your reputation, and the feedback loop that secures your future. Build the system, respect the user, and the intelligence will follow. Thank you."